

# Game Architecture Summary

## Project Overview

A Twin Peaks-inspired mystery board game with deckbuilding mechanics. Features a 24-hour circular board, dice rolling movement, event system, and character collection.

---

## File Structure

### Autoload Singletons (Global Scripts)

These are accessible from anywhere in the game via their node names.

**Location:** Project Settings → Autoload

#### GameState.gd

- **Purpose:** Manages global game state
- **Key Variables:**
  - `current_day: int` - Tracks which day the player is on
- **Signals:**
  - `day_changed(new_day: int)` - Emitted when a new day starts
- **Key Methods:**
  - `increment_day()` - Increases day counter and emits signal

#### TileEventManager.gd

- **Purpose:** Manages all tile-based events
- **Key Variables:**
  - `tile_events: Dictionary` - Maps hour numbers to event data
  - `event_pool: Array` - Pool of random events to spawn
- **Signals:**
  - `event_triggered(event_data: Dictionary)` - When an event activates
- **Key Methods:**
  - `setup_events()` - Creates events for the current day
  - `refresh_daily_events()` - Reshuffles events for a new day
  - `has_event(hour: int)` - Checks if an hour has an event
  - `get_event(hour: int)` - Returns event data for an hour

- `get_day_change_event(day_number: int)` - Creates day change popup data

### `CharacterDatabase.gd`

- **Purpose:** Stores all character definitions
  - **Key Variables:**
    - `characters: Dictionary` - All available characters by ID
  - **Key Methods:**
    - `setup_characters()` - Creates all character resources
    - `get_character(id: String)` - Returns specific character
    - `get_random_character()` - Returns random character
- 

## Resource Classes

### `Character.gd`

- **Type:** Resource (extends Resource)
  - **Purpose:** Data structure for character cards
  - **Properties:**
    - `character_name: String` - Character's full name
    - `title: String` - Character's role/title
    - `description: String` - Character background
    - `portrait: Texture2D` - Character portrait image
    - `rarity: String` - "common", "rare", or "legendary"
    - `insight: int` - Investigation stat (for future use)
    - `connection: int` - Town knowledge stat (for future use)
- 

## Scene Scripts

### `Board.gd` (Main Game Scene)

- **Attached to:** Board.tscn root node (Node2D)
- **Purpose:** Manages the circular board, player movement, and game flow

### Constants:

- `TILE_COUNT = 24` - Number of tiles (hours)

- `RADIUS = 300` - Outer radius of circle
- `INNER_RADIUS = 220` - Inner radius (creates donut shape)
- `START_HOUR = 12` - Hour at top of wheel (noon)
- `MOVE_DURATION_PER_TILE = 0.4` - Animation speed per tile

### Key Variables:

- `tiles: Array` - All tile nodes
- `player: Node2D` - Player token
- `player_tile_index: int` - Current position (0-23)
- `is_moving: bool` - Prevents input during movement
- `current_tween: Tween` - Active movement animation
- `event_popup: EventPopup` - Reference to popup UI

### Key Functions:

#### *Initialization:*

- `_ready()` - Sets up everything on game start
- `setup_ui()` - Configures buttons and labels
- `create_circular_board()` - Spawns all 24 tiles
- `spawn_player()` - Creates player token

#### *Board Creation:*

- `setup_tile_shape(tile, tile_index)` - Creates wedge geometry
- `setup_tile_signals(tile, tile_index)` - Connects click/hover events
- `add_hour_label(tile, hour, angle)` - Adds hour numbers
- `highlight_event_tiles()` - Colors borders of event tiles
- `highlight_tile_border(tile, tile_index)` - Makes border yellow/gold

#### *Visual Helpers:*

- `_create_wedge_points()` - Generates polygon points for tile shape
- `add_tile_border(tile, points)` - Adds Line2D border
- `get_time_of_day_color(hour)` - Returns gradient color for hour

#### *Player Movement:*

- `move_player_to(tile_index)` - Initiates movement to destination
- `calculate_path(from, to)` - Generates clockwise path

- `animate_along_path(path)` - Tweens player through tiles
- `get_tile_center_position(tile_index)` - Calculates world position

### *Signal Handlers:*

- `on_roll_button_pressed()` - Handles dice roll
- `on_tile_clicked()` - Handles tile click
- `on_tile_hover/unhover()` - Tile hover effects
- `on_move_finished()` - Called when movement completes
- `on_midnight_crossed()` - Triggers day change
- `on_day_changed(new_day)` - Updates day label
- `on_popup_closed()` - Resumes game after popup

### **CharacterCard.gd**

- **Attached to:** CharacterCard.tscn root node (PanelContainer)
- **Purpose:** Visual representation of a character card

### **Node References:**

- `name_label` - Character's full name
- `title_label` - Character's title/role
- `portrait_rect` - Portrait placeholder
- `description_label` - Character description

### **Key Methods:**

- `setup(character: Character)` - Populates card with character data

### **Visual Features:**

- Rarity-based color tinting (gray/blue/gold)
- Centered text layout
- Portrait placeholder for future images

### **EventPopup.gd**

- **Attached to:** EventPopup.tscn root node (CanvasLayer)
- **Purpose:** Displays event/day change popups

### **Node References:**

- `panel` - Main popup panel
- `title_label` - Event title
- `image_rect` - Event image (optional)
- `description_label` - Event description
- `close_button` - Button to dismiss

### Signals:

- `popup_closed` - Emitted when user closes popup

### Key Methods:

- `show_event(event_data: Dictionary)` - Displays an event
- 

## Scenes

### `Board.tscn`

- **Root:** Node2D
- **Children:**
  - 24 x Board\_tile instances (created at runtime)
  - Player instance (created at runtime)
  - CanvasLayer (UI layer)
    - Button (Roll Die button)
    - Label (Day counter, created at runtime)
  - EventPopup instance (created at runtime)

### `Board_tile.tscn`

- **Root:** Area2D (Tile)
- **Children:**
  - Polygon2D (visual wedge shape, created at runtime)
  - CollisionPolygon2D (for clicking, created at runtime)
  - Line2D (border, created at runtime)

### `Player.tscn`

- **Root:** Node2D
- **Children:**

- Sprite2D or ColorRect (player token visual)

#### **CharacterCard.tscn**

- **Root:** PanelContainer
- **Structure:**
  - MarginContainer
    - VBoxContainer
      - NameLabel
      - TitleLabel
      - PortraitRect (ColorRect)
      - DescriptionLabel (RichTextLabel)

#### **EventPopup.tscn**

- **Root:** CanvasLayer
- **Structure:**
  - ColorRect (dimming overlay)
    - Panel (popup container)
      - VBoxContainer
        - TitleLabel
        - ImageRect (TextureRect)
        - DescriptionLabel (RichTextLabel)
        - CloseButton

---

## Data Flow

### Game Start

1. **Autoloads initialize:**
  - **CharacterDatabase** creates all character resources
  - **TileEventManager** prepares event pool
  - **GameState** sets day to 1
2. **Board.\_ready() executes:**
  - Creates 24 tiles in a circle
  - Assigns gradient colors based on hour

- Highlights event tiles (yellow borders)
- Spawns player at hour 6
- Sets up UI (roll button, day counter)
- Creates event popup

## Dice Roll Flow

1. Player clicks "Roll Die" button
2. `_on_roll_button_pressed()`:
  - Rolls 1-6
  - Calculates destination tile
  - Calls `move_player_to()`
3. `move_player_to()`:
  - Generates clockwise path
  - Calls `animate_along_path()`
4. `_animate_along_path()`:
  - Creates tween animation
  - Moves player tile-by-tile
  - Checks for midnight crossing during path
  - If crosses midnight: pauses, shows day popup
5. `_on_move_finished()`:
  - Checks if landed on event tile
  - If yes: shows event popup
  - Re-enables roll button

## Day Change Flow

1. Player crosses from hour 23 → hour 0
2. `_on_midnight_crossed()`:
  - Pauses movement tween
  - `GameState.increment_day()`
  - `TileEventManager.refresh_daily_events()` - new random events
  - Clears old event highlights
  - Adds new event highlights
  - Shows day change popup
3. Player closes popup

4. `on_popup_closed()`:
  - Resumes movement tween
5. `on_day_changed()`:
  - Updates day counter label

## Event System Flow

1. **Setup (each day):**
    - `TileEventManager.refresh_daily_events()` called
    - Clears old events
    - Places lunch at hour 12 (always)
    - Randomly places 3-5 events from event pool
    - Returns event hours
  2. **Visual Highlighting:**
    - `Board.highlight_event_tiles()` called
    - Gets event hours from `TileEventManager`
    - Converts hours to tile indices
    - Changes tile borders to yellow/gold
  3. **Triggering:**
    - Player lands on tile
    - `on_move_finished()` checks current hour
    - If event exists: gets event data and shows popup
- 

## Key Coordinate Systems

### Tile Indexing

- **Tile Index (0-23):** Physical position in circle, starting at top (noon), going clockwise
  - Index 0 = hour 12 (noon, top)
  - Index 6 = hour 18 (6 PM, right side)
  - Index 12 = hour 0 (midnight, bottom)
  - Index 18 = hour 6 (6 AM, left side)

### Hour System

- **Hour (0-23):** What time is displayed on the tile



- Calculated as:  $\text{hour} = (\text{tile\_index} + \text{START\_HOUR}) \% \text{TILE\_COUNT}$
- Example: tile\_index 0 → hour 12 (noon at top)

## Conversion Formula

```
gdscript

# Tile index to hour:
hour = (tile_index + START_HOUR) % TILE_COUNT

# Hour to tile index:
tile_index = (hour - START_HOUR + TILE_COUNT) % TILE_COUNT
```

## Positioning Math

```
gdscript

# Angle for a tile index (in radians):
angle = (2 * PI / TILE_COUNT) * tile_index - PI / 2

# World position at tile center:
radius = (RADIUS + INNER_RADIUS) / 2.0
x = cos(angle) * radius
y = sin(angle) * radius
```

---

## Event Data Structure

Events are stored as Dictionaries with these keys:

```
gdscript

{
    "type": String,      # Event identifier
    "title": String,     # Display title
    "description": String, # Event text
    "image": String      # Path to image (optional, "" if none)
}
```

---

## Current Game Loop

1. Player starts at hour 6 (6 AM), Day 1
2. Click "Roll Die" → move 1-6 spaces clockwise

3. Player hops through each tile with animation
  4. If crosses midnight → pause, show day popup, continue
  5. If lands on event tile → show event popup
  6. Each new day → events reshuffle to new locations
  7. Repeat
- 

## **Future Integration Points**

### **Character Collection (Next Step)**

- Event popups will offer character cards
- Players can accept/decline characters
- Characters stored in player's deck
- Deck viewer UI to see collection

### **Not Yet Implemented**

- Card abilities/mechanics
- Deck strategy (hand size, drawing cards)
- Win/lose conditions
- Save/load system
- Character portraits
- Sound effects/music
- More complex event choices